

* Set ith Bit:-

eg.

	3	2	1	0	
N = 10	1	0	1	0	
i = 2	1	1	1	0	=> 14

Ans = $N \mid \text{magic number}$

magic number - a no. has only i^{th} bit set all other bits are unset.

$$2^i = (1 \ll i)$$

$$T.C: O(1)$$

$$S.C: O(1)$$

* Unset ith bit:-

eg.

	3	2	1	0	
N = 10	1	0	1	0	
i = 2	1	0	1	0	=> 10

$N = 23$

3	2	1	0
1	0	1	1

$i = 2$

1	0	0	1
---	---	---	---

 $\Rightarrow 19$

Tc: $O(1)$

Sc: $O(1)$

Super Number:-

i^{th} bit unset all other bits are unset:-

Ans $\Rightarrow N \neq$ Super number

Super number $= \sim (\text{magic number})$
 $= \sim (1 \ll i)$

* Check Bit:-

eg.

$N = 21$

4	3	2	1	0
1	0	1	0	1

$i = 2$

2 set

$N = 34$

5	4	3	2	1	0
1	0	0	0	1	0

$i = 3$

↑
unset

with right shift:-

$$(n \gg i) \& 1 == 1$$

with left shift:-

$$n \& (1 \ll i) \neq 0$$

★ Count bits:-

$$N = 10 \quad 1 \quad 0 \quad 1 \quad 0 \quad \Rightarrow 2$$

$$N = 27 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad \Rightarrow 4$$

Approach 1:-

$$T_c : O(1), S_c : O(1)$$

Approach 2:-

$$T_c : O(\log_2 N), S_c : O(1)$$

In worst case both approach will take 32 bit iterations

$$\rightarrow N = 2^{32} - 1$$

In second approach other case
2nd approach is better.

* Negative Number:-

32 bits

2^{31} 2^{30} 2^{29} 2^{28} ... 2^2 2^1 2^0



most significant
Bit MSB



least significant
bit LSB

To store negative number in our computer we consider MSB value to negative

To compute the negative of a numbers, we store it in it's 2's complement form.

8 bit	7	6	5	4	3	2	1	0
N=45	0	0	1	0	1	1	0	1
~ 45	1	1	0	1	0	0	1	0
+1	0	0	0	0	0	0	0	1
	1	1	0	1	0	0	1	1

1) Take inverse / negation of N

2) Add 1 to it

★ Ranges:-

32 bit $\rightarrow$ java, c++, go

32 bit

int $\rightarrow [-2 \times 10^9, 2 \times 10^9]$

64 bit c++, java, go

range of

long $\rightarrow [-9 \times 10^{18}, 9 \times 10^{18}]$

★ Importance of Constraints:-

constraints

$$1 \leq n \leq 10^6$$

$$1 \leq A_i \leq 10^6$$

worst case

10^5 terms

All largest 10^6

$$\text{maxSum} = 10^5 \times 10^6 = 10^{11}$$

Can we store 10^{11} in 32 bit int?

overflow